SESSION 9

Software Architecture Design and Design Patterns

AI Career Guidance & Skill Recommendation System

Project	AI Career Guidance & Skill Recommendation System
Deliverable	Architecture Design Document
Industry Mapping	Software Architecture and Design Patterns
Prepared by	Manya Goyal, B.Tech, KIET Group of Institutions
Team Members	Pawni Agrahari
Repository	[Add GitHub repository link]

Topics Covered in This Session

- Layered architecture design
- Client-server architecture
- Microservices (conceptual)
- Design patterns applied in the project

1. Architectural Goals

Goal	How the architecture supports it
Modifiability	Separate layers so UI, logic and data can change independently
Security	AI keys and database access only on the server; JWT checks at the API boundary
Scalability	Stateless API instances behind a load balancer; analytics isolated from the main flow
Testability	Services testable without UI or database through clear interfaces
Cost control	Single AI gateway with caching, retries and rate limits

2. Layered Architecture

The system uses a **four-layer architecture plus external resources**. A layer may call only the layer directly below it, which keeps dependencies one-directional.

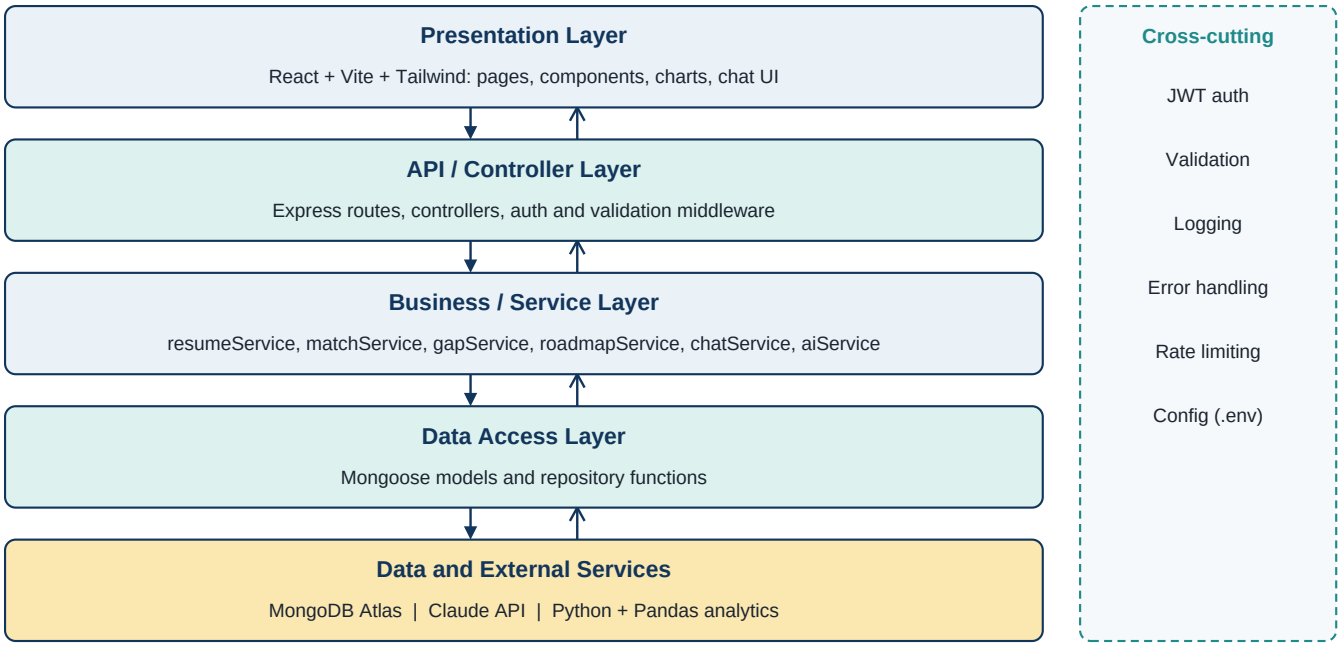

Figure 1: Layered architecture

Layer	Contents	Rules
Presentation	React pages (Dashboard, Upload, Recommendations, Roadmap, Chat, Admin), reusable components	No business rules; calls the API only
API / Controller	Express routes, controllers, middleware	Validates input, calls services, shapes responses
Business / Service	Matching, gap, roadmap, chat and AI services	All domain logic; no HTTP objects
Data Access	Mongoose schemas and queries	Only layer that talks to MongoDB
External	MongoDB Atlas, Claude API, Python analytics	Accessed through adapters

3. Client-Server Architecture

The application follows the **client-server** model. The React single-page application in the browser (client) sends HTTPS requests to the Express server, which applies business rules, reads and writes MongoDB, and calls external services.

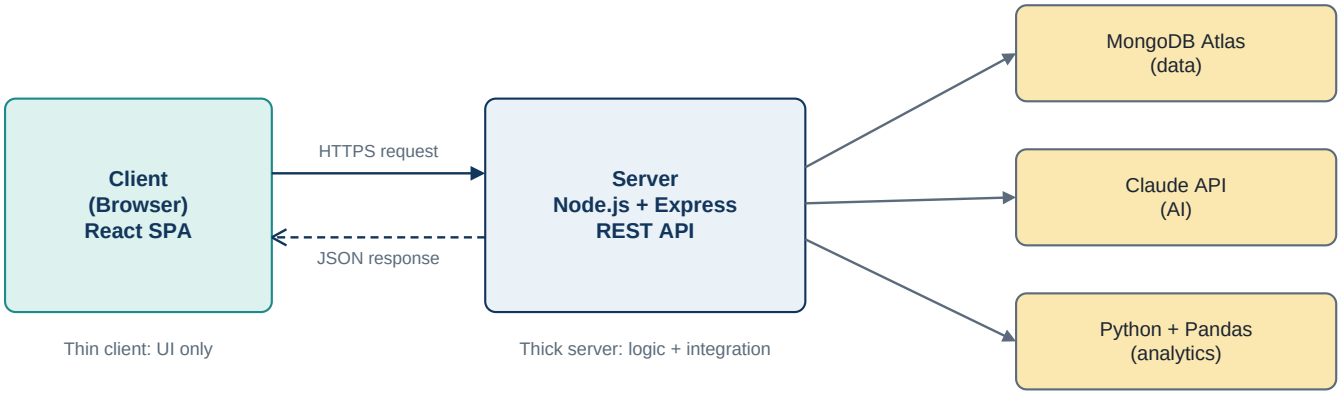

Figure 2: Client-server view and external services

- **Why it fits:** clear separation, many students (clients) share one set of careers and courses on the server.
- **Security:** the browser never sees the Claude API key or database credentials.

- **Deployment:** client on a static host (for example Vercel or Netlify), server on a Node host (for example Render), database on MongoDB Atlas.

4. Microservices Architecture (Conceptual)

In a microservices design each business capability becomes a small, independently deployable service with its own data store, reached through an API gateway. This is a **conceptual target** for future growth; the first release is a modular monolith whose modules (Session 8) already match these service boundaries.

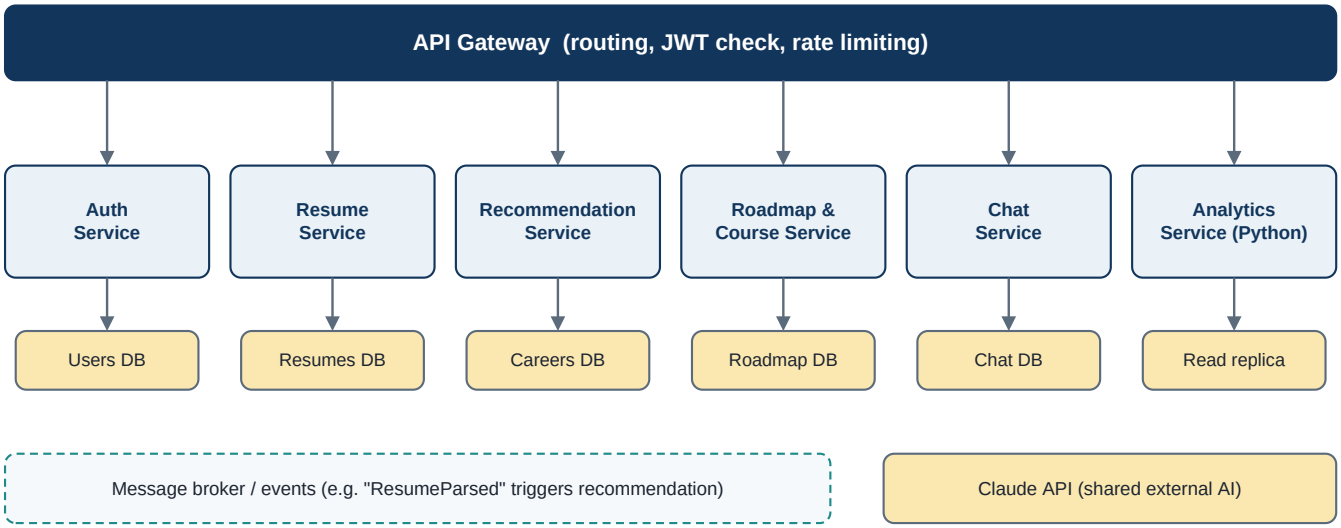

Figure 3: Conceptual microservices decomposition

Aspect	Current: modular monolith	Future: microservices
Deployment	One Express app, easy to build and debug	Each service deployed and scaled separately
Data	One MongoDB database, separate collections	Database per service
Communication	Function calls between modules	REST or events through a broker
Team fit	Small team, one semester	Larger teams owning services
Main cost	Scaling is all-or-nothing	Operational complexity, distributed debugging

Decision: start as a layered modular monolith (low effort, fast delivery) and extract Analytics and AI services first if load grows, because they have the heaviest and most independent workload.

5. Design Patterns Used

Pattern	Category	Where used	Benefit
MVC	Architectural	Mongoose models, Express controllers, React views	Separates data, logic and presentation
Layered	Architectural	Overall backend structure	One-directional dependencies
Repository / DAO	Structural	Data access functions around Mongoose models	Business code independent of database details
Facade	Structural	RecommendationService hides matching, gap and explanation steps behind one call	Simple interface for controllers
Adapter	Structural	aiService wraps the Claude API in our own interface	Provider can be replaced without changing callers
Singleton	Creational	Database connection and Claude client created once and reused	Saves resources, consistent configuration
Factory	Creational	Parser factory selects PDF or DOCX text extractor by file type	New file types added without changing callers

Pattern	Category	Where used	Benefit
Strategy	Behavioural	Interchangeable scoring strategies (weighted, keyword, AI-assisted)	Matching algorithm can change at runtime
Chain of Responsibility	Behavioural	Express middleware: auth, validation, rate limit, error handler	Each step has one job and can be reordered
Observer	Behavioural	React state and hooks update the UI when roadmap progress changes	UI stays in sync with data

Example: Adapter and Strategy in pseudocode

```
class AiService {                       // Adapter around Claude API
  analyzeResume(text)  { return this.client.ask(RESUME_PROMPT, text); }
  generateRoadmap(gaps){ return this.client.ask(ROADMAP_PROMPT, gaps); }
}

const scoring = { weighted: weightedScore, keyword: keywordScore };   // Strategy
const score = scoring[config.strategy](profile, career);
```

6. Technology Mapping and Deployment

Component	Technology	Hosting (suggested)
Client	React, Vite, Tailwind CSS, charts library	Vercel / Netlify
Server	Node.js, Express, Mongoose, JWT, Multer	Render / Railway
Database	MongoDB Atlas	Atlas free tier
AI	Claude API through aiService	Anthropic API
Analytics	Python, Pandas scheduled script writing aggregates to MongoDB	Same server host or scheduled job

7. Quality Attribute Check

Attribute	Architectural tactic	Requirement
Performance	Cache AI results per resume; index MongoDB lookups	NFR-01
Security	JWT middleware, HTTPS, server-side keys, input validation	NFR-03, NFR-04
Reliability	Retry and timeout in aiService; fallback roadmap	NFR-05
Maintainability	Layers, modules, adapter and facade patterns	NFR-08
Scalability	Stateless API, separable analytics service	NFR-02